Imam Abdulrahman Bin Faisal University
Deanship of Preparatory Year and Supporting Studies
Computer Department

Basics of Programming through Python
Conditional Statements



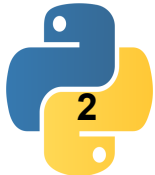
Introduction to Programming

COMP106

Term 1 | 2026-2027

Learning outcomes

- Understand the concept and usage of selection structure and conditional statements.
- Know various types of conditional statements available in Python (if, elif, Nested if).
- Handling an exception using try, except else and finally.
- Analyse the problem, decide, and evaluate conditions.



Types of control structures

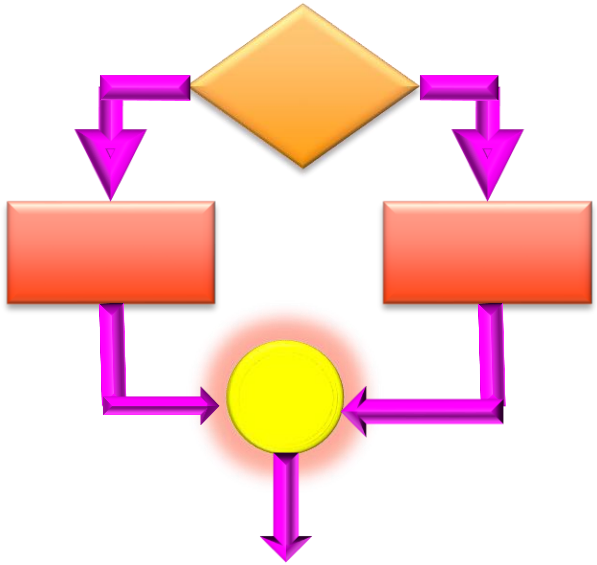
A Structured programming is an important feature of a programming language which comprises following logical structure:

1. SEQUENCE
2. SELECTION
3. ITERATION OR LOOPING



Selection Structure

A selection statement causes the program control to be transferred to a specific flow based upon whether a certain condition is true or not.



Introduction to conditional statements

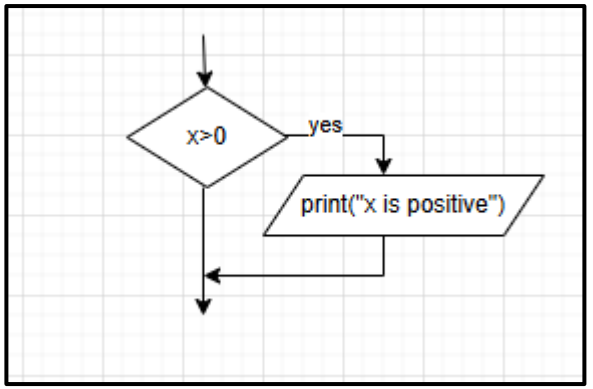
- Conditional statements are used to control the flow of the program.
- Conditional statements in Python perform different computations or actions depending on whether a specific Boolean constraint evaluates to true or false.
- if, elif, and else are the conditional statements in Python.



Conditional Execution: if statement

- *Conditional statements* check conditions and change the behavior of the program accordingly.
- The simplest form is the ***if statement***: (one selection statement)

```
if x > 0 :
    print('x is positive')
```



Conditional Execution: if statement

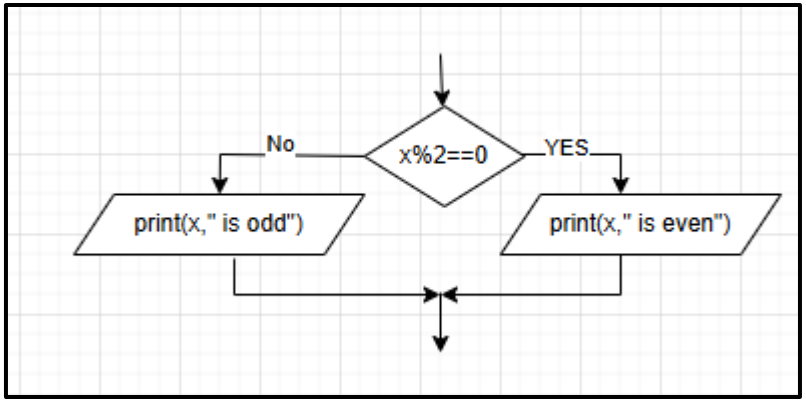
- An “if statement” is written by using the if keyword.
- Python if statement is used for decision-making operations.
- It contains a body of code which runs only when the condition given in the if statement is True. If the condition is False, then the optional else statement runs which contains some code for the **else** condition.



Alternative Execution: if-else

- There are two possibilities, and the condition determines which one gets executed. (two-way selection statement)

```
if x%2 == 0 :
    print(x,' is even')
else :
    print(x,' is odd')
```



if –then-else Logic

if - else statements

- There are a few important items to remember about **if statements:**
- The colon (:) is important and essential.
- The header of the compound statement (i.e. if statement) is isolated from the body.
- Python uses indentation for blocks and nested blocks.
- All rows indented after the colon will be executed whenever the Boolean expression is valid.

Syntax of if-else statement

```
if condition:
        statement_T
else:
        statement_F
```



if–else :Example (1)

```
age = 15
if (age >=18):
    print ("Elegible for Voting")
else:
    print("Not Eligible for Voting")
print ("Statement after if statement")
```



```
Not Eligible for Voting
Statement after if statement
```

if- else: Example (2)

- Find the output of this code?

```
A=15
B=10
if B>A:
    print("B is greater than A")
else:
    print("A is greater than B")
```



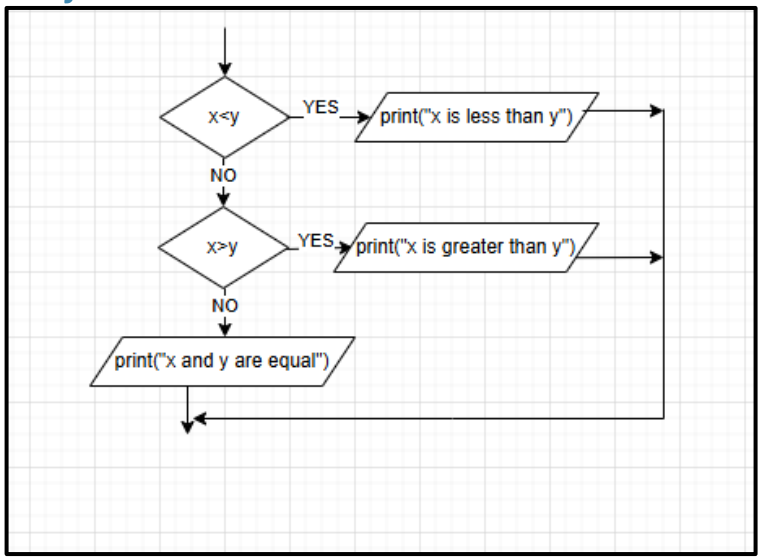
A is greater than B



Chained Conditionals (if- elif Statement)

- Sometimes, there are more than two possibilities, and we need more than two branches (Multiple-Way).

```
if x < y:
  print('x is less than y')
elif x > y:
  print('x is greater than y')
else:
  print('x and y are equal')
```



If-Then-Elself Logic

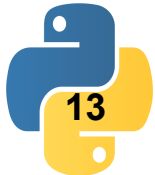
if- elif Statement

- The elif keyword is python way of saying :"if the previous conditions were not true, then try this condition.
- elif is abbreviation of else if.
- There is no limit of the number of elif statement.

  but only a single final else is allowed and it must be the last branch in the statement.

Syntax of if-elif statement

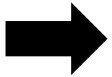
```
if CONDITION 1:
        Statements _A
elif CONDITION 2:
        Statement _B
else:
        Statement _C
```



if - elif: Example (1)

- Find the output of this code:

```
Age = 27
if Age >= 60:
    print ('Senior Discount')
elif Age <=18:
    print ('No Discount')
else:
    print ('Junior Discount')
```



Junior Discount

if - elif: Example (2)

- Find the output of this code:

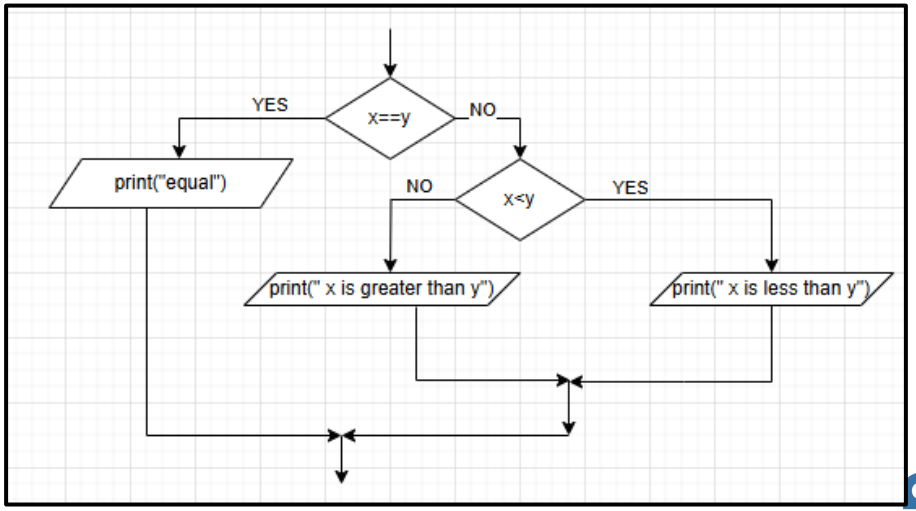
```
A=33
B=33
if B>A:
    print("B is greater than A")
elif A>B:
    print("A is greater than B")
else:
    print("A and B are equals")
```



Nested Conditionals

- One conditional can also be nested within another:

```
if x == y:
   print('equal')
else:
   if x < y:
        print('x is less than y')
   else:
        print('x is greater than y')
```



Nested If Statements

Nested if Statement---syntax

```
if  EXPRESSION  1:
        STATEMENT(S)
        if EXPRESSION 2:
              STATEMENT(S)
         elif EXPRESSION 3:
              STATEMENT(S)
         elif EXPRESSION 4:
              STATEMENT(S)
         else:
               STATEMENT(S)
else:
        STATEMENT(S)
```



Nested if...Example

- Find the output of the code below.
- Rewrite the code using if—elif statement.

```
1  # find if the number is zero, positive or negative
2  num=15
3  if num>=0:
4      if num==0:
5          print("zero")
6      else:
7          print("Positive number")
8  else:
9      print("Negative number")
```



Comparison Operators

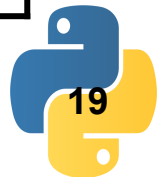
- **Boolean expressions** ask a question and produce a **Yes** or **No** result which we use to control program flow

- **Boolean expressions** using comparison operators to evaluate to **True / False** or **Yes / No**

- **Comparison operators** look at variables but **do not change** the variables

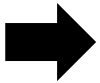
Python	Meaning
<	Less than
<=	Less than or Equal to
==	Equal to
>=	Greater than or Equal to
>	Greater than
!=	Not equal
is	The same as
is not	Not the same as

Remember: "=" is used for assignment.



Comparison Operators

```
x = 5
if x == 5 :
   print('Equals 5')
if x > 4 :
  print('Greater than 4')
if  x >= 5 :
   print('Greater than or Equals 5')
if x < 6 : print('Less than 6')
if x <= 5 :
   print('Less than or Equals 5')
if x != 6 :
   print('Not equal 6')
```



```
Equals 5
Greater than 4
Greater than or Equals 5
Less than 6
Less than or Equals 5
Not equal 6
```

Logical Operators

- There are three logical operators: and, or, and not.
  - x > 0 and x < 10 → is true only if x is greater than 0 and less than 10.
  - n%2 == 0 or n%3 == 0 → is true if either one of the conditions is true, that is, if the number is divisible by 2 or 3.
  - not (x > y) → is true if x > y is false; that is, if x is less than or equal to y.

Any nonzero number is interpreted as "True."

17 and True → True



Truth table

A	B	A and B	A or B	not A
False	False	False	False	True
False	True	False	True	True
True	False	False	True	False
True	True	True	True	False



Python Exceptions Handling

- Python provides two very important features to handle any unexpected error in your Python programs and to add debugging capabilities in them:
  - Exception Handling
  - Assertions

What is Exception?

- An exception is an event that stops a program's normal flow, when a Python script encounters a situation that it can't cope with, it raises an exception.
- An exception is a Python object that represents an error.

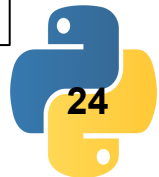


Handling an exception

- If you have some *suspicious* code that may raise an **exception**, you can defend your program by placing the suspicious code in a try: block.
- After the try: block, include an except: statement, followed by a block of code which handles the problem as elegantly as possible.

Syntax

```
try:
    You do your operations here;
    ......................
except:
    If there is Exception, then
    execute this block.
```



Catching exceptions using try and except

- Here is a sample program to convert a Fahrenheit temperature to a Celsius temperature:

```
inp = input('Enter Fahrenheit Temperature: ')
fahr = float(inp)
cel = (fahr - 32.0) * 5.0 / 9.0
print(cel)
```

- If we execute this code and give it invalid input, it simply fails with an unfriendly error message

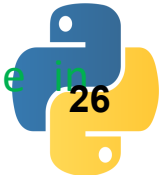


Catching exceptions using try and except

```
inp = input('Enter Fahrenheit Temperature:')
try:
  fahr = float(inp)
  cel = (fahr - 32.0) * 5.0 / 9.0
  print(cel)
except:
   print('Please enter a number')
```

- If an exception (error) occurs in the try block, Python jumps out of the try block and executes the sequence of statements in the except block.

- You can use **try & except** with any script code.
- You can write try: at a beginning of your code or anywhere else.
- You can write any script code in **except** not just print.



Try and Except: Find the output?

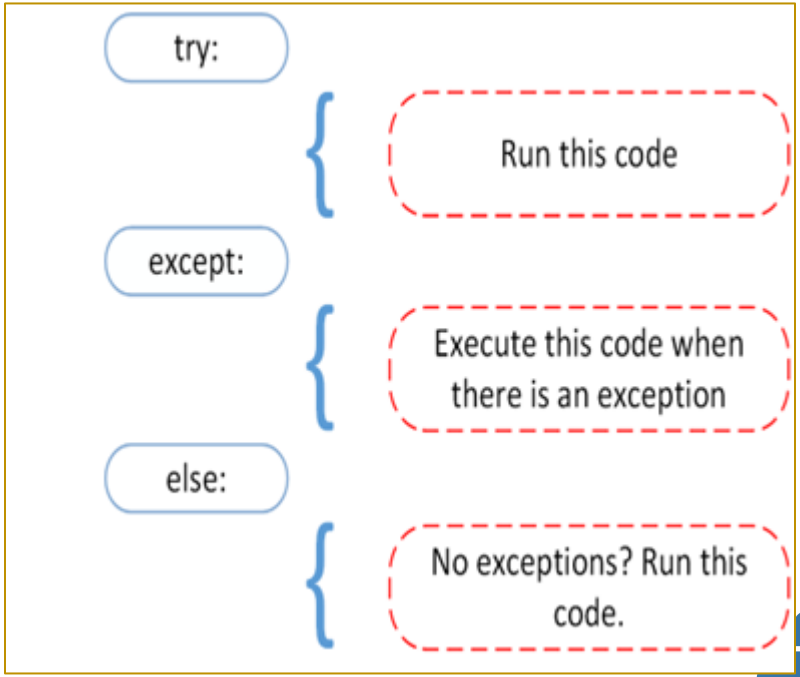
```
try:
    Var1 = int(input("Enter the first number: "))
    Var2 = input("Enter the second number: ")
    print(Var1 + Var2)
except:
    print("What are you doing? you cannot add an integer with string")
```

```
Enter the first number: 10
Enter the second number: 12
What are you doing? you cannot add an integer with string
```



try except else

- After the except clause, you can include an else clause. The code in the else-block executes if the code in the try: block does not raise an exception.
- The else-block is a good place for code that does not need the try: block's protection.

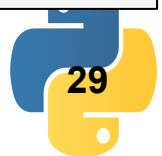


Try and check

Find the output if:

- Var 1=6 and Var 2=3
- Var 1=6 and Var 2=12
- Var 1=6 and Var 2=0
- Var 1=0 and Var 2=2

```
Var1=int(input("enter the first number"))
Var2=int(input("enter the second number"))
try:
        # Floor Division : Gives only Fractional
        # Part as Answer
        result = Var1/Var2
except ZeroDivisionError:
        print("Sorry ! You are dividing by zero "
else:
        print("Yeah ! Your answer is :", result)
```

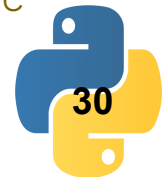


The Try-Finally Clause

- You can use a finally: block along with a try: block.
- The finally block is a place to put any code that must execute, whether the try-block raised an exception or not.

The syntax of the try-finally statement is this:

```
try:
    You do your operations here;
    ......................
    Due to any exception, this may be skipped.
finally:
        This would always be executed.
        ......................
```



Try and check

Find the output when:

- name=Ahmed
- name=Emy
- name=Lee
- name=Zaineb

```
1
2  name=input("enter your name")
3  try:
4      x=name[3]
5      print("char at index 3 is",x)
6      print("no exception")
7  except:
8      print("index error")
9  else:
10     print("else block is executed because no exception")
11 finally:
12     print("finally will always executes")
```

Short-circuit evaluation of logical expressions

- Short-circuiting means stopping the evaluation of a logical expression early because the final result is already clear.

```
x = 6
y = 0
x >= 2 and (x/y) > 2
Error
```

y=0, which causes a runtime error (division by zero)

```
x = 1
y = 0
x >= 2 and (x/y) > 2
False
```

The first part of these expressions x >= 2 evaluated to False so the (x/y) was not ever executed.

(short-circuit)



Short-circuit evaluation of logical expressions

- A guard evaluation can be strategically placed before the evaluation that might cause an error.

```
x = 1
y = 0
x >= 2 and y != 0 and (x/y) > 2
```

→ False

```
x = 6
y = 0
x >= 2 and y != 0 and (x/y) > 2
```

→ False

```
x = 6
y = 0
x >= 2 and (x/y) > 2 and y != 0
```

→ Error (without guard evaluation)



Any Questions?

Thank you for listening ☺

